Random Forest

Sangmyung Ha

May 10, 2024

Outline

References

- Trevor Hastie, Robert Tibshirani, Jerome Friedman (2009), "The Elements of Statistical Learning (ESL)", Chapter 9.2, 15.1, 15.2, 15.3

Decision Trees

- **Regression**
  Given,

$$\{(x_i, y_i)\}_{i=1}^{n},\ x_i \in \mathbb{R}^p, y_i \in \mathbb{R}$$

  we want to estimate a model $T : \mathbb{R}^p \to \mathbb{R}$ such that $T(x_{n+1})$ is a good predictor for y_{n+1}.

- **Classification**
  Given,

$$\{(x_i, y_i)\}_{i=1}^{n},\ x_i \in \mathbb{R}^p, y_i \in \{1, \ldots, J\}$$

  we want to estimate a classifier $C : \mathbb{R}^p \to \{1, \ldots, J\}$ such that $C(x_{n+1})$ is a good predictor for y_{n+1}.

Decision Trees: Regression

Let $f(x_i) = \mathbb{E}[y_i|x_i]$ so that we may write

$$y_i = f(x_i) + e_i$$

Suppose the conditional expectation function $f(x)$ can be well approximated by a simple function. In other words, there exists a partition $R_1, \ldots, R_M$ of $\mathbb{R}^p$ such that

$$f(x) \approx \sum_{m=1}^{M} c_m 1_{\{x \in R_m\}}$$

The parameters to estimate in this model is the pair $\{(R_m, c_m)\}_{m=1}^{M}$. How do we estimate them?

Decision Trees: Regression

As with least squares, we write

$$f(x) = \sum_{m=1}^{M} c_m 1_{\{x \in R_m\}}$$

and find the pair $\{(R_m, c_m)\}_{m=1}^{M}$ that jointly minimizes the squared error loss function:

$$\sum_{i=1}^{n} (y_i - f(x_i))^2 = \sum_{m=1}^{M} \sum_{x_i \in R_m} (y_i - c_m)^2$$

Comparison with linear regression

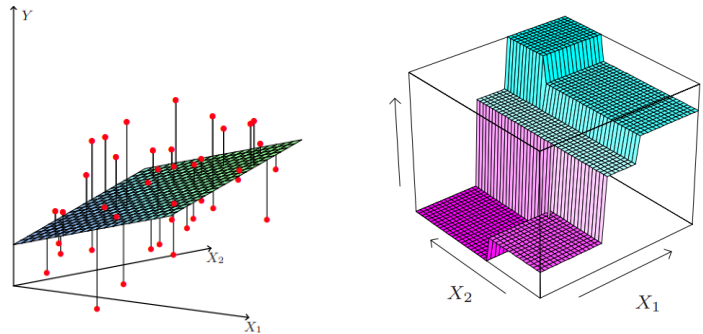

Figure 1: Figure 3.1 and 9.2 of ESLII

In linear regression, the loss function to be minimized is the squared deviation from a linear model. In decision trees, the loss function to be minimized is squared deviation from a simple function.

Decision Trees: Regression

Given $(R_m)_{m=1}^M$, the value c_m that minimizes the squared error loss

$$\sum_{i=1}^{n}(y_i - f(x_i))^2 = \sum_{m=1}^{M}\sum_{x_i \in R_m}(y_i - c_m)^2$$

is given by the sample average $\hat{c}_m = \frac{1}{N_m}\sum_{x_i \in R_m} y_i$ for the region R_m, where N_m is the number of samples in R_m. Thus, the problem reduces to finding the partition $\{R_m\}_{m=1}^M$ that minimizes

$$\sum_{m=1}^{M}\sum_{x_i \in R_m}(y_i - \hat{c}_m)^2$$

There are infinite number of ways to partition $\mathbb{R}^p$. How do we find this?

Decision Trees: Regression

- If M is greater than n, we could make the loss zero by assigning one observation to each region. There are multiple solutions.
- If M is less than n, instead of partitioning $\mathbb{R}^p$, we may first try partitioning $(x_i)_{i=1}^n$ into M sets. However, the number of ways to partition n elements into M sets is very large and it would be computationally infeasible to try all the possible combinations. Thus, we allow only *binary splits* and proceed with the *greedy algorithm*.

Decision Trees: Regression

Greedy Algorithm:

As with forward stepwise regression, even if the goal is to partition $(x_i)_{i=1}^n$ into M sets, we first start with the whole set and consider partition into two sets.

1. We first split $(x_i)_{i=1}^n$ into two sets $(x_i)_{i\in S_1}$ and $(x_i)_{i\in S_2}$.
2. Then, in the subsequent step of finding the best partition into three sets, we split either $(x_i)_{i\in S_1}$ or $(x_i)_{i\in S_2}$ into two sets.
3. Suppose $(x_i)_{i\in S_2}$ has been split into two sets $(x_i)_{i\in S_{21}}$ and $(x_i)_{i\in S_{22}}$, then we are left with partition into three sets $(x_i)_{i\in S_1}$, $(x_i)_{i\in S_{21}}$, and $(x_i)_{i\in S_{22}}$.
4. We proceed in these steps until we have partitioned $(x_i)_{i=1}^n$ into M sets.

Note:

- Once two samples x_i and x_j are separated, they will never fall into a same set in the subsequent steps.
- The above algorithm does not guarantee finding the optimal partition.

Decision Trees: Regression

Binary Splits:

We define binary splits as splitting the samples $(x_i) \in \mathbb{R}^p$ based only on whether one covariate exceeds a specific threshold or not. In splitting a set, we only consider binary splits and exhaustively consider all the possible binary splits to search for a best splitting point. Suppose we consider splitting the j-th variable at a split point s. Define the pair of half-planes

$$R_1(j, s) = \{x_i | x_{ij} < s, i = 1, \ldots, n\}$$
$$R_2(j, s) = \{x_i | x_{ij} \geq s, i = 1, \ldots, n\}$$

We then seek the splitting variable j and split point s that solve:

$$\min_{j,s} \left[\sum_{x_i \in R_1(j,s)} (y_i - \hat{c}_1)^2 + \sum_{x_i \in R_2(j,s)} (y_i - \hat{c}_2)^2 \right]$$

where $\hat{c}_1$ and $\hat{c}_2$ are averages in the respective regions.

Decision Trees: Algorithm

Algorithm

Recursively repeat the following steps for each set in the partition:

(i) Pick the best split-point among each variable.

(ii) Compute the decrease in split criterion for each variable using the split point in step (i).

(iii) Select the variable which results in the maximum decrease in split-criterion

(iv) Split the node into two daughter nodes using the variable selected in (iii) at the split point selected in (i).

- The splitting criterion to be used in the regression trees is the squared error loss.
- When a new sample falls in a specific node, predictions are made by the average of the samples in the node.

Decision Trees: Classification

Decision trees can naturally be extended to classification. Recall that in the multinomial logit model, conditional choice probability is given as

$$\mathbb{R}\{y = j | X = x\} = \frac{e^{f_j(x)}}{\sum_{k=1}^{K} e^{f_k(x)}}$$

where $f_j(x)$ is the decision function for j-th class. Suppose that all the decision functions $f_j(x)$ can be well approximated by simple functions. In other words, there exists a partition $R_1, \ldots, R_m$ of $\mathbb{R}^p$ such that

$$f_j(x) \approx \sum_{m=1}^{M} c_{mj} 1_{\{x \in R_m\}}$$

for all $j = 1, \ldots, J$. How do we estimate the parameters $\{(R_m, (c_{mj})_{j=1}^{J})\}_{m=1}^{M}$?

Decision Trees: Classification

As with linear classification model (multinomial logit), we write

$$f_j(x) = \sum_{m=1}^{M} c_{mj} 1_{\{x \in R_m\}}$$

and find the parameters $\{(R_m, (c_{mj})_{j=1}^{J})\}_{m=1}^{M}$ that jointly maximizes the log likelihood function:

$$\log\left(\prod_{i=1}^{n}\prod_{j=1}^{J}\left(\frac{e^{f_j(x_i)}}{\sum_{k=1}^{K} e^{f_k(x_i)}}\right)^{1_{\{y_i=j\}}}\right) = \sum_{i=1}^{n}\sum_{j=1}^{J}\log\left(\left(\frac{e^{f_j(x_i)}}{\sum_{k=1}^{K} e^{f_k(x_i)}}\right)^{1_{\{y_i=j\}}}\right)$$

$$= \sum_{m=1}^{M}\sum_{x_i \in R_m}\sum_{j=1}^{J}\log\left(\left(\frac{e^{c_{mj}}}{\sum_{k=1}^{K} e^{c_{mk}}}\right)^{1_{\{y_i=j\}}}\right)$$

Decision Trees: Classification

For a given R_m, the values $c_{mj}, j = 1, \ldots, J$ that maximizes the log likelihood has a closed form solution. To see this, note that

$$\begin{aligned}
&\sum_{m=1}^{M} \sum_{x_i \in R_m} \sum_{j=1}^{J} \log \left(\left(\frac{e^{c_{mj}}}{\sum_{k=1}^{K} e^{c_{mk}}} \right)^{1_{\{y_i = j\}}} \right) \\
&= \sum_{m=1}^{M} \sum_{x_i \in R_m} \sum_{j=1}^{J} 1_{\{y_i = j\}} \log \left(\frac{e^{c_{mj}}}{\sum_{k=1}^{K} e^{c_{mk}}} \right) \\
&= \sum_{m=1}^{M} \sum_{j=1}^{J} \left(\sum_{x_i \in R_m} 1_{\{y_i = j\}} \right) \log \left(\frac{e^{c_{mj}}}{\sum_{k=1}^{K} e^{c_{mk}}} \right)
\end{aligned}$$

Decision Trees: Classification

Let $\hat{p}_{mj}$ denotes the proportion of samples in the region R_m with label $y = j$. Then, we may write

$$\hat{p}_{mj} = \frac{1}{N_m} \sum_{x_i \in R_m} 1_{\{y_i = j\}}$$

where N_m is the number of samples x_i in R_m. Thus,

$$\begin{aligned} &= \sum_{m=1}^{M} \sum_{j=1}^{J} \left(\sum_{x_i \in R_m} 1_{\{y_i = j\}} \right) \log \left(\frac{e^{c_{mj}}}{\sum_{k=1}^{K} e^{c_{mk}}} \right) \\ &= \sum_{m=1}^{M} \sum_{j=1}^{J} N_m \hat{p}_{mj} \log \left(\frac{e^{c_{mj}}}{\sum_{k=1}^{K} e^{c_{mk}}} \right) \end{aligned}$$

Note that the maximizing the above log-likelihood can be solved for each m individually.

Decision Trees: Classification

Let's focus only on R_m:

$$\sum_{j=1}^{J} N_m \hat{p}_{mj} \log\left(\frac{e^{c_{mj}}}{\sum_{k=1}^{K} e^{c_{mk}}}\right) = \sum_{j=1}^{J} N_m \hat{p}_{mj} \left[c_{mj} - \log\left(\sum_{k=1}^{K} e^{c_{mk}}\right)\right]$$

FOC for c_{ml} is given by:

$$N_m \hat{p}_{ml} - \sum_{j=1}^{J} N_m \hat{p}_{mj} \frac{1}{\sum_{k=1}^{K} e^{c_{mk}}} e^{c_{ml}} = 0$$

which may be rewritten as

$$\begin{aligned} \hat{p}_{ml} &= \left(\sum_{j=1}^{J} \hat{p}_{mj}\right) \frac{e^{c_{ml}}}{\sum_{k=1}^{K} e^{c_{mk}}} \\ &= \frac{e^{c_{ml}}}{\sum_{k=1}^{K} e^{c_{mk}}} \end{aligned}$$

Decision Trees: Classification

This means that the conditional choice probability that maximizes the log-likelihood is given by

$$\frac{e^{\hat{c}_{ml}}}{\sum_{k=1}^{K} e^{\hat{c}_{mk}}} = \hat{p}_{ml}$$

for $l = 1, \ldots, M$. This is intuitive. If we want to maximize the likelihood using a constant decision function, we want to equate the estimated conditional probability with the in-sample proportion.

Decision Trees: Classification

Also, the concentrated objective function is given by

$$\sum_{m=1}^{M} N_m \sum_{j=1}^{J} \hat{p}_{mj} \log \left(\frac{e^{\hat{c}_{mj}}}{\sum_{k=1}^{K} e^{\hat{c}_{mk}}} \right) = \sum_{m=1}^{M} N_m \sum_{j=1}^{J} \hat{p}_{mj} \log \hat{p}_{mj} \tag{1}$$

and maximizing the (1) w.r.t. $(R_m)_{m=1}^{M}$ is equivalent to maximizing the log-likelihood jointly w.r.t. $(R_m)_{m=1}^{M}$ and $((\hat{c}_{mj})_{j=1}^{J})_{m=1}^{M}$.

Decision Trees: Classification

Instead of finding the partition $(R_m)_{m=1}^M$ so that the concentrated log-likelihood

$$\sum_{m=1}^{M} N_m \sum_{j=1}^{J} \hat{p}_{mj} \log \hat{p}_{mj}$$

is maximized, we may instead find $(R_m)_{m=1}^M$ so that the loss function

$$-\sum_{m=1}^{M} N_m \sum_{j=1}^{J} \hat{p}_{mj} \log \hat{p}_{mj}$$

is minimized. Here,

$$-\sum_{j=1}^{J} \hat{p}_{mj} \log \hat{p}_{mj}$$

is called *cross-entropy* loss function.

Decision Trees: Algorithm

Finding the partition $(R_m)_{m=1}^M$ is done in the exactly same way as in regression trees, by binary splits and greedy algorithm.

Algorithm

Recursively repeat the following steps for each set in the partition:

(i) Pick the best split-point among each variable.

(ii) Compute the decrease in split criterion for each variable using the split point in step (i).

(iii) Select the variable which results in the maximum decrease in split-criterion

(iv) Split the node into two daughter nodes using the variable selected in (iii) at the split point selected in (i).

- The splitting criterion to be used in the classification trees is the cross-entropy loss.
- When a new sample falls in a specific node, predictions are made by the majority vote in that node, $\arg\max_j \hat{p}_{mj}$.

Decision Trees

Properties of Decision Trees:

- Easy to interpret
- Easily ignores redundant variables
- High variance, low prediction accuracy

Random Forest and Bagging

- *Bootstrap*: From $\{(x_i, y_i)\}_{i=1}^n$, generate 'new' data by resampling n samples with replacement.
- Bootstrap aggregation, or *bagging*, is a model averaging stragegy. In bagging, many models are trained over each of bootstrapped samples, and the final predictions calculated by averaging over the predictions made by each models. Bagging works especially well for high-variance and low-bias procedures, such as trees.
- *Random Forest* is a modification of bagging that builds a collection of 'de-correlated' trees. Here, trees are 'de-correlated' by searching over only a randomly selected subset of variables when finding the best split points.
- Typically, $m = \sqrt{p}$ number of features are selected in each node in classification and $m = p/3$ are used in the regression.

Random Forest: Algorithm

Algorithm

1. For $b = 1$ to B
   (a) Draw a bootstrap sample $(x_i^*)_{i=1}^n$ from the training data.
   (b) Grow a random-forest tree T_b on the bootstrapped data, by recursively repeating the following steps for each terminal node of the tree.
      (i) Select m variables at random from the p variables.
      (ii) Pick the best variable/split-point among the chosen variables.
      (iii) Split the node into two daughter nodes.
2. Output the ensemble of trees $\{T_b\}_1^B$

To make a prediction at a new point x,

- Regression: $\hat{f}_{rf}^B(x) = \frac{1}{B}\sum_{b=1}^B T_b(x)$
- Classification: $\hat{C}_{rf}^B =$ majority vote$\{\hat{C}_b(x)\}_1^B$

Variable Importance: Decrease in Loss (Impurity)

At each split in each tree, the improvement in the split criterion is accumulated over all the trees in the forest separately for each variable. This accumulation can be used as a measure of variable importance.

- Variables for which split has been made multiple times will have large variable importance.
- Fast to compute.
- Features that have high explanatory power do not necessarily have high predictive power. A feature might have a high variable importance measure due to overfitting.
- This is a biased approach, as it has a tendency to inflate the importance of continuous features or high-cardinality categorical variables.